

Java Inheritance and Implementation

Sample Programs and UML Diagrams for All 8 Cases

Case	Declaration	Status
1	C1 extends C2	Valid
2	C1 extends C2, C3	Invalid
3	C1 implements I1	Valid
4	C1 implements I1, I2	Valid
5	C1 implements C2 extends I1	Invalid
6	I1 extends I2	Valid
7	I1 implements C1	Invalid
8	I1 extends I2, I3	Valid

Important Java rule: A class can extend only one class, but it can implement multiple interfaces. An interface can extend one or more interfaces.

Quick Reference of Rules

Syntax	Valid?	Reason
class C1 extends C2	Yes	A class may extend one class.
class C1 extends C2, C3	No	Java does not support multiple class inheritance.
class C1 implements I1	Yes	A class may implement an interface.
class C1 implements I1, I2	Yes	A class may implement multiple interfaces.
class C1 implements C2 extends I1	No	implements requires interfaces and extends must come before implements
interface I1 extends I2	Yes	An interface may extend another interface.
interface I1 implements C1	No	An interface cannot implement a class.
interface I1 extends I2, I3	Yes	An interface may extend multiple interfaces.

Case 1: C1 extends C2

Status: VALID

Rule / Explanation

A class can extend exactly one class. C1 becomes a subclass of C2 and inherits accessible members from C2.

Sample Java Program

```
class C2 {  
    void showC2() {  
        System.out.println("This is C2");  
    }  
}  
  
class C1 extends C2 {  
    void showC1() {  
        System.out.println("This is C1");  
    }  
}  
  
public class C1ExtC2 {  
    public static void main(String[] args) {  
        C1 obj = new C1();  
  
        obj.showC1();  
        obj.showC2();  
    }  
}
```

Expected Output

```
This is C1  
This is C2
```

UML Diagram - Single Class Inheritance

C1 (Class) — extends —> C2 (Class)

Case 2: C1 extends C2, C3

Status: INVALID

Rule / Explanation

Invalid in Java. A class cannot extend two or more classes. Java does not support multiple inheritance of classes because it can create ambiguity when parent classes contain the same members.

Sample Java Program

```
class C2 {
    void display() {
        System.out.println("This is C2");
    }
}

class C3 {
    void display() {
        System.out.println("This is C3");
    }
}

class C1 extends C2, C3 {
    void show() {
        System.out.println("This is C1");
    }
}

public class Case2 {
    public static void main(String[] args) {
        System.out.println("C1 extends C2, C3 is invalid in Java.");
    }
}
```

Note: The class declaration shown above is intentionally invalid and will produce a compiler error. The main method is a separate runnable demonstration of the rule.

Expected Output

```
The declaration class C1 extends C2, C3 is not valid Java syntax.
A class can extend only one class.
```

UML Diagram - Multiple Class Inheritance (Invalid)

C1 (Class) — extends —> C2 (Class)
C1 (Class) — extends —> C3 (Class) — INVALID

Case 3: C1 implements I1

Status: VALID

Rule / Explanation

A class can implement one interface. The class must provide implementations for all abstract methods declared by the interface, unless the class itself is abstract.

Sample Java Program

```
interface I1{
    int x=100;
    void dispaly();
    void show();
}
class a implements I1{
    @Override
    public void dispaly(){
        System.out.println("it is from interface");
    }
    public void show(){
        System.out.println("method from interface");
    }
}
public class C1ImpI1 {

    public static void main(String[] args) {
        a obj=new a();
        obj.dispaly();
        obj.show();
        System.out.println(obj.x);
    }
}
```

Expected Output

```
it is from interface
method from interface
100
```

UML Diagram - Class Implementing One Interface

a (Class) - - - - - > I1 (Interface) [implements]

Case 4: C1 implements I1, I2

Status: VALID

Rule / Explanation

A class can implement multiple interfaces. It must implement the required abstract methods from all implemented interfaces, provided there are no unresolved method conflicts.

Sample Java Program

```
interface I1 {
    void method1();
}

interface I2 {
    void method2();
}

class C1 implements I1, I2 {
    public void method1() {
        System.out.println("Method from I1");
    }

    public void method2() {
        System.out.println("Method from I2");
    }
}

public class Case4 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
    }
}
```

Expected Output

```
Method from I1
Method from I2
```

UML Diagram - Multiple Interface Implementation

C1 (Class) - - - > I1 (Interface)

C1 (Class) - - - > I2 (Interface)

Case 5: C1 implements C2 extends I1

Status: INVALID (original case declaration)

Rule / Explanation

The original case statement is invalid because a class cannot implement another class. The legal combined form is class C1 extends C2 implements I1. Your supplied program uses the legal pattern and additionally implements I2 and I3.

Sample Java Program — supplied replacement

```
interface I1 {
    void method1();
}

interface I2 {
    void method2();
}

interface I3 {
    void method3();
}

class C2 {
    void show() {
        System.out.println("This is C2");
    }
}

class C1 extends C2 implements I1, I2, I3 {

    public void method1() {
        System.out.println("I1 method");
    }

    public void method2() {
        System.out.println("I2 method");
    }

    public void method3() {
        System.out.println("I3 method");
    }
}

public class C1ExtC2ImpI1I2I3 {
    public static void main(String[] args) {

        C1 obj = new C1();

        obj.show();           // From C2
        obj.method1();        // From I1
        obj.method2();        // From I2
        obj.method3();        // From I3
    }
}
```

Expected Output — for the supplied replacement code

```
This is C2
I1 method
I2 method
I3 method
```

UML Diagram - Legal Combined Form Used by Supplied Code

C1 (Class) — extends —> C2 (Class)

C1 (Class) - - - > I1, I2, I3 (Interfaces)

Case 6: I1 extends I2

Status: VALID (interface-to-interface inheritance)

Rule / Explanation

An interface can extend another interface. Your supplied program uses I2 extends I1; therefore a class implementing I2 must satisfy the methods inherited from I1 as well as the methods declared in I2.

Sample Java Program — supplied replacement

```
interface I1{
    int x=20;
    void show();
    void dispaly();
}
interface I2 extends I1{
    int y=50;
    void show();
    void play();
}
class A implements I2{
    public void show(){
        System.out.println("it is implements from interface");
    }
    public void play(){
        System.out.println("y value from I2"+y);
    }
    public void dispaly(){
        System.out.println("implemts from i1");
    }
}

public class InterfacetoInterface {

    public static void main(String[] args) {
        A obj=new A();
        obj.dispaly();
        obj.show();
        obj.play();
        System.out.println(obj.x);
    }
}
```

Expected Output — for the supplied replacement code

```
implemts from i1
it is implements from interface
y value from I250
20
```

UML Diagram - Interface Inheritance Used by Supplied Code

I2 (Interface) — extends —> I1 (Interface)

A (Class) - - - > I2 (implements)

Case 7: I1 implements C1

Status: INVALID

Rule / Explanation

Invalid in Java. An interface does not implement a class. An interface extends one or more interfaces. A class implements one or more interfaces.

Sample Java Program

```
class C1 {
    void display() {
        System.out.println("This is C1");
    }
}

interface I1 implements C1 {
    void show();
}

public class Case7 {
    public static void main(String[] args) {
        System.out.println("I1 implements C1 is invalid in Java.");
        System.out.println("An interface can extend interfaces, not implement classes.");
    }
}
```

Expected Output

```
I1 implements C1 is invalid in Java.
An interface can extend interfaces, not implement classes.
```

UML Diagram - Interface Implementing Class (Invalid)

I1 (Interface) - - - > C1 (Class) — INVALID

Valid relationship: C1 - - - > I1 (class implements interface)

Case 8: I1 extends I2, I3

Status: VALID

Rule / Explanation

An interface can extend multiple interfaces. This is how Java supports multiple inheritance of interface contracts while avoiding multiple inheritance of classes.

Sample Java Program

```
interface I2 {
    void method2();
}

interface I3 {
    void method3();
}

interface I1 extends I2, I3 {
    void method1();
}

class C1 implements I1 {
    public void method1() {
        System.out.println("Method from I1");
    }

    public void method2() {
        System.out.println("Method from I2");
    }

    public void method3() {
        System.out.println("Method from I3");
    }
}

public class Case8 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.method1();
        obj.method2();
        obj.method3();
    }
}
```

Expected Output

```
Method from I1
Method from I2
Method from I3
```

UML Diagram - Multiple Interface Inheritance

I1 (Interface) — extends —> I2 (Interface)
I1 (Interface) — extends —> I3 (Interface)

UML / Relationship Summary

Case	Relationship	Result
1	C1 extends C2	Single class inheritance
2	C1 extends C2, C3	Invalid multiple class inheritance
3	C1 implements I1	One interface implementation
4	C1 implements I1, I2	Multiple interface implementation
5	C1 extends C2 implements I1, I2, I3	Legal combined form used in supplied code
6	I2 extends I1 (supplied code)	Interface inheritance
7	I1 implements C1	Invalid interface/class relationship
8	I1 extends I2, I3	Multiple interface inheritance

Key Relationships

- extends is used for class inheritance and interface inheritance.
- implements is used when a class implements an interface.
- A class can extend only one class, but it can implement multiple interfaces.
- An interface can extend one or more interfaces.

Execution Steps

1. Open Eclipse IDE or any Java IDE.
2. Create a Java project named InheritanceDemo.
3. For each valid case, create a Java file using the class name shown in the program.
4. For invalid cases, keep the intentionally invalid declaration only for studying the compiler rule; run the separate demonstration program.
5. Run the program and compare the console output with the expected output in this PDF.
6. Use the UML diagrams to understand the relationship between classes and interfaces.
7. Capture console screenshots if your assignment requires them.

Conclusion

Java supports single inheritance between classes and multiple inheritance through interfaces. The `extends` keyword is used for class inheritance and interface inheritance, while `implements` is used by classes to implement interfaces. The eight cases demonstrate the valid and invalid combinations.

Note on the supplied replacements: Case 5 uses the legal form `class C1 extends C2 implements I1, I2, I3`. Case 6 uses `interface I2 extends I1`. The outputs shown for those pages are calculated from the exact code supplied in the request.